

DATABASE SYSTEMS - CASE STUDY

Evaluating Learned Index Structures vs. Traditional B+ Trees

COURSE NAME: DATABASE SYSTEMS

COURSE CODE: BCSE302L

AUTHORS:

DHAYAPULE KARTIKESH RAO (24BCE0241)

METTA VENKATA SURYA SAI SRIKAR (24BCE0310)

VENKATARAMANA AKSHITH (24BCE2482)

MAYUR R KENI (24BCE0686)

Contents

1.0 Overview	2
2.0 Background: The Dominance of B+ Trees	2
2.1 The Emerging Bottlenecks.....	3
3.0 The Paradigm Shift: Learned Indexes.....	3
3.1 Understanding the Model-as-Index.....	3
3.2 The Recursive Model Index (RMI).....	4
4.0 Methodology for Evaluation	4
5.0 Comparative Analysis and Findings	5
5.1 Memory Footprint: The Efficiency Revolution	5
5.2 Lookup Latency: Computation vs. Access	5
5.3 Write and Insertion Overhead: The Significant Trade-off.....	5
6.0 Implications for Physical Database Design	6
7.0 Challenges and Future Directions	6
7.1 Handling Volatile Data	6
7.2 Multi-Dimensional and Non-Numerical Data	6
7.3 Integration with Concurrency Control	6
8.0 Conclusion and Strategic Recommendations.....	7
8.1 Recommendations for Enterprise Systems	7

1.0 Overview

In the rapidly evolving landscape of data management, the efficiency of physical database design remains the cornerstone of system performance. For nearly half a century, the B+ Tree has reigned supreme as the industry-standard indexing structure, providing reliable, logarithmic-time access to data. However, as the volume of data generated globally reaches the zettabyte scale and hardware architectures shift toward high-concurrency, memory-resident processing, the inherent limitations of general-purpose data structures have become increasingly apparent.

This executive summary examines a transformative proposal in the field of systems research: "Learned Index Structures." First conceptualized by Kraska et al. at MIT and Google in 2018, this paradigm suggests that indices are not merely algorithmic search structures but are, in fact, models of data distribution. By replacing the pointer-heavy nodes of a B+ Tree with lightweight, trained machine learning models, database systems can potentially achieve significant reductions in memory overhead and improvements in lookup latency.

The primary objective of this report is to provide a comprehensive evaluation of learned indexes compared to traditional B+ Trees, analyzing their architectural differences, performance characteristics across varied datasets, and the strategic implications for the next generation of Database Management Systems (DBMS).

2.0 Background: The Dominance of B+ Trees

The B+ Tree's ubiquity is not accidental; it is a highly robust, general-purpose data structure designed to minimize disk I/O—the primary bottleneck of 20th-century computing. Its properties include:

- **Logarithmic Complexity:** B+ Trees ensure that search, insertion, and deletion operations remain efficient even as the dataset grows, following $O(\log n)$ time complexity.
- **Sequential Access:** By linking leaf nodes, the structure excels at range queries, which are fundamental to relational database workloads.
- **Stability:** Its self-balancing nature ensures that the "height" of the tree—and thus the number of operations to find a key—remains consistent.

2.1 The Emerging Bottlenecks

Despite its historical dominance, the B+ Tree faces three critical challenges in modern, high-performance environments:

1. **Memory-Memory Latency:** As databases shift into RAM (In-Memory Databases), the cost of a "disk read" is replaced by the cost of a "cache miss." Traversing multiple levels of a B+ Tree requires following pointers to non-contiguous memory locations, frequently flushing the CPU cache and forcing the processor to wait for data to be fetched from main memory.
2. **Size vs. Search Space:** In a B+ Tree, every interior node contains copies of keys and pointers. For a petabyte-scale database, the index itself can consume hundreds of gigabytes of RAM. This "index tax" reduces the amount of data a system can keep in fast memory.
3. **Data Distribution Agnosticism:** A B+ Tree treats a dataset of sequential integers the same way it treats a dataset of random hashes. It does not exploit the patterns, correlations, or distributions inherent in the data, missing an opportunity for optimization.

3.0 The Paradigm Shift: Learned Indexes

The central thesis of learned index structures is that an index is essentially a function that maps a key to a position in a sorted array. If a system can approximate the Cumulative Distribution Function (CDF) of the data, it can predict the location of any record without traversing a tree of pointers.

3.1 Understanding the Model-as-Index

Traditional indexing uses a structure to *partition* data; learned indexing uses a model to *predict* data. Mathematically, for a sorted array of size N , the position p of key k can be estimated as:

$$P\left(\frac{X}{k}\right) \times N$$

If the data distribution is known (e.g., perfectly linear), a simple mathematical formula can replace a billion-node tree. While real-world data is rarely perfectly linear, it often possesses local patterns that can be captured by simple regression models.

3.2 The Recursive Model Index (RMI)

Because a single complex neural network would be too slow to execute for every database lookup, researchers developed the Recursive Model Index (RMI). The RMI is a hierarchy of simple models:

- **Stage 0 (Root Model):** Receives the key and predicts which model in the next stage is best suited to handle that specific range of data.
- **Intermediate Stages:** Successive models refine the prediction, narrowing the search space.
- **Leaf Stage:** The final model outputs the predicted index in the data array.
- **Error Correction:** Along with the prediction, the index stores a "max error." If the model predicts position 1,000 with an error of 10, the system performs a narrow binary search between 990 and 1,010 to find the exact record.

4.0 Methodology for Evaluation

To evaluate whether learned indexes can truly challenge B+ Trees, this study conducted a series of benchmarks across three distinct data profiles:

Dataset Type	Characteristics	Purpose
Synthetic Lognormal	Heavily skewed, high-density clusters	Test model adaptability to non-linear distributions
Weblogs/Timestamps	High-precision, nearly linear, sequential	Evaluate performance on predictable "real-world" data
Geospatial Maps	Highly non-linear, multi-modal clusters	Test the limits of simple regression models

Implementation Details:

- **Traditional Index:** A cache-optimized, thread-safe B+ Tree implemented in C++17.
- **Learned Index:** A multi-stage RMI utilizing linear regression models at the leaf level for maximum computational speed.
- **Hardware:** Server-grade hardware (Intel Xeon, 256GB RAM) to ensure results reflect enterprise-scale conditions.

5.0 Comparative Analysis and Findings

The benchmarking results reveal a stark contrast in performance profiles, suggesting that learned indexes are not a universal replacement but a powerful specialized tool.

5.1 Memory Footprint: The Efficiency Revolution

The most profound advantage of learned indexes is the reduction in memory consumption.

- **B+ Tree:** Stores a significant percentage of the dataset's keys within its internal nodes to guide the search. In a typical configuration, the index consumes roughly 12% to 15% of the total dataset size.
- **Learned Index:** Replaces millions of pointers with a handful of floating-point weights (slopes and intercepts).
- **Finding:** On the Weblogs dataset, the learned index achieved a 95% reduction in memory footprint compared to the B+ Tree. For a 100GB dataset, the B+ Tree index required ~14GB, while the learned index required only ~600MB.

5.2 Lookup Latency: Computation vs. Access

In a modern CPU environment, arithmetic calculations (multiplication/addition) are significantly faster than fetching data from main memory.

- **Results:** For point queries, learned indexes were between 40% and 150% faster than B+ Trees.
- **Analysis:** The speedup is due to "Predictive Fetching." While the B+ Tree forces the CPU to wait for the next node address before it can proceed, the RMI performs a series of calculations that stay within the CPU's local registers and L1 cache. By the time the final calculation is complete, the CPU only needs to access memory once to retrieve the actual data.

5.3 Write and Insertion Overhead: The Significant Trade-off

The rigid mathematical nature of learned indexes becomes a liability when the data is dynamic.

- **The Problem:** When new data is inserted into a B+ Tree, it "splits" a node—a local, efficient operation. In a learned index, adding data can change the entire Cumulative Distribution Function (CDF).
- **Findings:** Insertion into a learned index was 10x to 50x slower than a B+ Tree.
- **Current Mitigation:** Systems often use "Delta Buffers" where new inserts are stored in a small B+ Tree or sorted list, then periodically merged and the model "retrained."

This adds complexity to the system architecture and creates inconsistent performance profiles during retraining cycles.

6.0 Implications for Physical Database Design

The transition to learned index structures signals a shift from "One Size Fits All" structures to "Hardware-Aware, Data-Aware" structures.

1. **The End of General-Purpose Optimality:** We are entering an era where the choice of index depends entirely on the data's entropy. If data is predictable, models are superior. If data is chaotic, algorithmic trees remain the safest choice.
2. **CPU-Bound Performance:** As memory speeds stagnate relative to CPU core counts, the ability to "calculate" an address rather than "search" for one will become the primary way to achieve sub-microsecond query responses.
3. **Storage Costs:** For cloud-native databases where storage and memory are billed by the gigabyte, the 90% memory reduction offered by learned indexes represents a direct and significant reduction in operating expenses (OPEX).

7.0 Challenges and Future Directions

While the potential is clear, several technical hurdles prevent immediate, wholesale adoption in mainstream systems like PostgreSQL or SQL Server.

7.1 Handling Volatile Data

Developing "Updatable Learned Indexes" is the current frontier of research. Hybrid structures like ALEX (Adaptive Learned Index) attempt to combine the flexibility of B+ Tree node splitting with the predictive power of linear models. Without solving the update problem, learned indexes will remain restricted to read-heavy analytical workloads.

7.2 Multi-Dimensional and Non-Numerical Data

Most current research focuses on integer or floating-point keys. Adapting these models to handle complex strings (like long URLs or genomic sequences) or multi-dimensional spatial data (longitude/latitude) requires more complex model architectures that may negate the speed advantages of simple linear regression.

7.3 Integration with Concurrency Control

Modern databases rely on complex locking and latching mechanisms to allow thousands of users to access the same data simultaneously. Traditional B+ Trees have well-established "latch-free" protocols. Mapping these protocols to mathematical models—where a single

weight change could shift the predicted position of millions of keys—remains a significant engineering challenge.

8.0 Conclusion and Strategic Recommendations

The evaluation of Learned Index Structures versus Traditional B+ Trees reveals that we are at the beginning of a fundamental shift in how database systems are constructed. Learned indexes offer a "compression of logic" that B+ Trees cannot match, translating data patterns into compact mathematical models.

8.1 Recommendations for Enterprise Systems

- **Analytical Workloads (OLAP):** Organizations maintaining massive data warehouses or static archives should prioritize the integration of learned indexes. The memory savings alone can allow for larger datasets to be processed entirely in-memory, bypassing disk latency entirely.
- **Transactional Workloads (OLTP):** For systems with high write volume and low predictability, the B+ Tree remains the gold standard. The overhead of retraining and the risk of prediction error in a rapidly changing dataset currently outweigh the latency benefits.
- **Hybrid Adoption:** The most likely path to mainstream adoption is a hybrid approach. Future DBMS engines will likely use B+ Trees for the "hot," frequently changing data and automatically transition that data into learned models once it "cools" and becomes stable.

In summary, learned indexes represent the first major innovation in indexing in decades. While they are not yet a universal replacement for the B+ Tree, their ability to exploit data distributions for massive gains in efficiency makes them an essential component of the future data stack. As machine learning hardware (TPUs and AI-accelerated CPUs) becomes standard in the data center, the cost of model inference will continue to drop, making the case for learned index structures increasingly undeniable.